Job Scraping + Visa Intelligence Engine – Current Workflow (Python / Django)

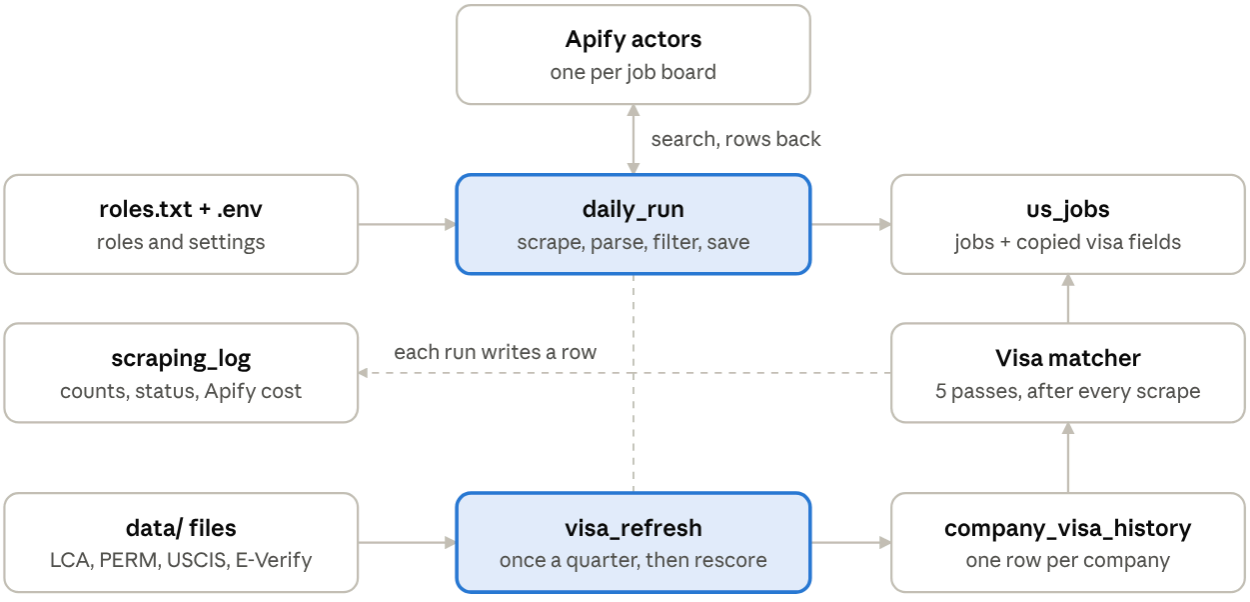
Oct 30, 2026 · @Sai

Summary

One daily command scrapes the 8 required US job boards for every role, saves the jobs to PostgreSQL, and tags each job with its company's government visa record (H-1B, PERM / green card, E-Verify, grade A–F). The system now runs as a Django app inside the client's existing Django project, with no separate server.

- **Jobs (Component 1A):** LinkedIn, Indeed, Glassdoor, Dice, Monster, CareerBuilder, ZipRecruiter and JobRight, plus optional remote-job boards. Roles come from a roles file, and each run takes up to a set number of new jobs per role from each board, posted within the last 24 hours.
- **Visa data (Component 1B):** DOL LCA (H-1B), DOL PERM (green card), the USCIS H-1B Employer Data Hub and the E-Verify employer list are loaded into one record per company, then scored 0–100 and graded A–F.
- **Matching (Component 1C):** every job is linked to its company's visa record through five passes (hand-checked aliases, exact name, fuzzy, prefix, last-resort exact) and the visa fields are copied onto the job.

Two pipelines fill two tables, and the matcher joins them



System architecture · daily and quarterly pipelines

The daily pipeline fills `us_jobs` and the quarterly pipeline fills `company_visa_history`. The matcher joins them after every scrape, and every scrape, match and visa load writes a row to `scraping_log`.

What changed from the Node.js version

The scraping, visa loading, matching and scoring rules are unchanged: the same settings, tables, SQL, parsing and grading. What changed is how it is installed and run.

Area	Node.js version	Python / Django version
Runs as	A separate Node.js project	A Django app (`job_scraper`) inside the existing Django project
Commands	`npm run daily-run` , `npm run visa-refresh` , …	`python manage.py daily_run` , `python manage.py visa_refresh` , …
Database connection	`DATABASE_URL` and `DB_POOL_MAX` in `.env`	The project's own `DATABASES` setting (`JOB_SCRAPER_DB_ALIAS` picks the entry)
Table setup	`npm run setup-db`	`python manage.py setup_db` (or `migrate job_scraper`)
Parallel scraping	Each role waits for all its boards before the next role starts	Optional shared queue: all role × board searches share one queue (`SCRAPE_SHARED_QUEUE=true`)
Apify cost	Not recorded	Recorded per search in `scraping_log` and totalled at the end of every run
Company aliases	35 seeded	39 seeded, still added by hand with `add_alias`
Code comments and README	Long, written around the Node code	Short, plain notes only where needed; README rewritten for Python

An API to start a scrape is planned for a later phase. The functions the commands use can be called directly from Python for that (see Code layout).

Setup

The app needs Python 3.9 or newer, Django 4.2 or newer and a PostgreSQL database (the SQL uses PostgreSQL features such as `ON CONFLICT` and full-text search). Setup takes eight steps.

1. **Copy the app.** Put the `job_scraper` folder into the Django project, next to the other apps.
2. **Install the requirements** inside the project's virtual environment:

   ```
   pip install -r job_scraper/requirements.txt
   ```

   This installs Django, a PostgreSQL driver (psycopg2 or psycopg 3), apify-client, openpyxl, python-dateutil and python-dotenv.
3. **Register the app** in `settings.py`: add `'job_scraper'` to `INSTALLED_APPS`.
4. **Choose the database.** If the project's `default` database is PostgreSQL, nothing more is needed. Otherwise add a PostgreSQL entry to `DATABASES` and point the app at it:

   ```
   DATABASES = {
       'default': {...},
       'jobs': {'ENGINE': 'django.db.backends.postgresql', 'NAME':
   'jobplatform', ...},
   }
   JOB_SCRAPER_DB_ALIAS = 'jobs'
   DATABASE_ROUTERS = ['job_scraper.routers.JobScraperRouter']
   ```

5. **Create the tables:**

   ```
   python manage.py setup_db
   ```

   This runs `schema.sql`: it creates the 4 tables and their indexes and loads the 39 hand-checked company aliases. It is safe to run again at any time. It never deletes or overwrites data, and it adds any new columns to existing tables, so it is also the step to run after updating the app. `python manage.py migrate job_scraper` does the same thing.
6. **Settings.** Copy `job_scraper/env.example` to `.env` in the folder `manage.py` is run from, and fill in `APIFY_API_TOKEN`. Every setting is explained in that file (see Settings below). The real `.env` holds the Apify token and is never shared.
7. **Roles and government files.** Put these in the same folder as `manage.py`:
   - `roles.txt`: the roles to search, one per line (`#` lines and blank lines are skipped)
   - `data/`: the government files, in `data/lca`, `data/perm`, `data/uscis-hub` and `data/everify`
8. **Load the visa data and check the install:**

   ```
   python manage.py visa_refresh
   python manage.py test_board_parsers
   python manage.py daily_run --check-config
   ```

`test_board_parsers` runs offline parser checks and should end with "All parser checks passed." `--check-config` prints the settings and the run-size estimate without spending anything on Apify.

Windows. Activate the virtual environment with `venv\Scripts\activate` before every command (the prompt then starts with `(venv)`). If the log shows broken characters, run `set PYTHONUTF8=1` first, or `setx PYTHONUTF8 1` once to make it permanent.

Time zone. Dates without a zone (`started_at`, `posting_date`, `first_seen_at` and the others) are stored in the local time of the machine running the commands, which follows `settings.TIME_ZONE` (Asia/Kolkata in the current project).

Daily run

The whole daily job is one command, `python manage.py daily_run`. It runs these steps in order and never deletes any job or scraped data.

1. **Load and check settings.** All limits come from `.env` and can be overridden for one run on the command line. If any value is invalid, the run stops before scraping anything. The settings are printed at the start and end of the run, each with where it came from (command line, `.env` or default).
2. **Read the roles.** Roles come from `roles.txt` (or a `.json` file). Duplicates are removed ignoring upper / lower case and extra spaces, so "Data Analyst" and "data analyst" count as one role.
3. **Print a run-size estimate.** Roles × boards to search, the maximum number of jobs and the worst-case number of billed Apify rows, printed before any money is spent.
4. **Scrape every role on every board.** For each role and board the scraper:
   a. asks the board's Apify actor for jobs matching the role, posted within the freshness window;
   b. converts each row to the standard job shape with that board's own parser;
   c. drops jobs whose title does not match the role, jobs outside the US and jobs older than the freshness window;
   d. saves each remaining job as new, already known (refreshed) or a cross-board duplicate;
   e. pages deeper only when it still needs more new jobs, up to a cost ceiling;
   f. writes one row to `scraping_log` with the counts, the Apify run id, the number of Apify runs and their cost.
5. **Retry blocked boards.** A board that returns nothing because it was blocked or broke (for example a bot check) is retried once after 5 seconds. Every other board keeps running.

6. **Run the visa matcher** once all roles are done (see Visa matching).
7. **Mark stale jobs inactive.** Jobs not seen by any scrape for 45 days get `is_active = FALSE`. The rows are kept, and a job that is scraped again becomes active again.
8. **Print the summary:** jobs per role and per board against the limit; a status table with one row per role and board (counted, limit, fetched, new, refreshed, duplicates, stale, title mismatches, pages, status, cost and a note on why the board gave what it gave); totals per board (counted / limit, fetched, cost); and the total Apify cost of the run.

Two test modes use the same command: `--check-config` shows the settings and estimate without scraping, and `--each-board` runs a small test (3 jobs) on every board and prints one result table.

Parallel searches and the shared queue

With `SCRAPE_LIMIT_SCOPE=per_board`, every board has its own limit, so board searches run side by side, up to `SCRAPE_PARALLEL_BOARDS` at once (8 in the current `.env`). Starts are spaced 1.5 seconds apart so the actors don't all start in the same second. `SCRAPE_SHARED_QUEUE` decides how roles share those slots:

`SCRAPE_SHARED_QUEUE`	How the searches run
`true` (current)	All role × board searches go into one queue. A new search starts as soon as any search finishes, whatever its role, so a slow board never holds up the other roles.
`false`	One role at a time. Its boards run in parallel, and the next role starts only when that role's slowest board is done.

The saved jobs and the end-of-run tables are the same either way; the shared queue only finishes sooner. With `SCRAPE_LIMIT_SCOPE=per_role`, the boards share one limit, so they always run one after another in `SCRAPE_SOURCES` order.

Error handling

One failure never stops the whole run. Each problem is kept to the smallest unit it affects and explained in the log and the end-of-run status table.

Situation	What happens
A board's actor is not set in `.env`	That board is skipped for the run, with one message naming the `.env` line to add. The other boards run normally.
A board is blocked or breaks (bot-check page, HTTP 403, actor crash, parser failure)	The board is retried after 5 seconds (`SCRAPE_RETRY_FAILED`, default once). If it still fails, it is marked FAILED for that role and the run continues.
A board returns nothing	The scraper reads the actor's own log and states the reason: no listings in the time window, every job removed by the title filter, or blocked.
A board returns placeholder rows that are not jobs	The run is treated as empty and the reason is taken from the actor's log.
One job fails to save	The error is logged for that job only; the rest of the page is still saved.
A search or a whole role fails	It is logged and recorded as failed with zero jobs, and the other searches and roles still run.
A setting in `.env` or on the command line is invalid	The run stops before any scraping, with a message naming the bad value and what is allowed. Unknown command-line options also stop the run.
A government file is missing	That source is skipped with a message giving its download page; the other files still load.
The visa load fails	The error is written to `scraping_log` as a failed `visa_refresh` run.

Every board search, successful or not, writes a row to `scraping_log` with its status and any error message, so failures can be reviewed after the run.

Job scraping (Component 1A)

Each board is scraped through an Apify actor set only in `.env` (`APIFY_*_ACTOR`); no actor is hardcoded. If a board's actor line is missing, that board is skipped with a message naming the line to add, and the other boards still run.

Board	Apify actor	How it is searched	Notes
LinkedIn, Indeed, Glassdoor	`flash_scraper/multi-jobboard-scraper`	Role keywords, last 24 hours	One shared actor and parser. The title filter runs inside the actor, so off-title rows are not billed.
Dice	`solidcode/dice-scraper`	Role, posted within 1 day	Relies on Dice's own title search
Monster	`unfenced-group/monster-scraper`	Role, posted within 1 day, whole US	Detail fetching is on, because without it company and location come back empty
CareerBuilder	`ahmed_jasarevic/careerbuilder-scraper`	One call per role using CareerBuilder's "today" search URL	Asks only for the role's limit in one call, so rows are never re-read and re-billed. Needs a role of more than one word.
ZipRecruiter	`memo23/ziprecruiter-scraper`	Role as an exact phrase, 1 day	Needs a US residential proxy; foreign jobs are dropped; one call per role, sized to the whole fetch budget
JobRight	`jobscrawler/jobright-scraper`	Role, last 24 hours	Must run without a proxy to avoid a Cloudflare block

Board	Apify actor	How it is searched	Notes
Optional remote boards	Same multi-board actor	Remotive, Jobicy, Himalayas, HN Hiring, RemoteOK, We Work Remotely, Working Nomads	Can be switched off in `SCRAPE_SOURCES`; The Muse and DevITjobs are off by default

Roles and limits. Roles are plain title searches. Each run takes up to `SCRAPE_MAX_JOBS_PER_ROLE` new jobs per role from each board (`per_board` mode). With `SCRAPE_COUNT_MODE=inserted`, only brand-new jobs count toward the limit; jobs already saved and cross-board duplicates are refreshed but not counted, and the scraper pages deeper to find new ones. A cost ceiling of `SCRAPE_MAX_FETCH_MULTIPLIER` × the limit stops any one board from paging forever. A different limit can be set for any single board, for example `glassdoor:30`.

Freshness. Jobs must be posted within the last 24 hours (`SCRAPE_FRESHNESS_DAYS=1`). Every board is asked for its own 24-hour filter. A second check in our code compares whole days, because most boards give a posting date but no time, so it keeps today's and yesterday's jobs. Jobs with no posting date are kept (`SCRAPE_KEEP_UNDATED=true`), because the board already filtered them by age.

Title matching. On boards whose search is loose (the multi-board sites, CareerBuilder, ZipRecruiter, JobRight), a job is kept only if its title contains the role's distinctive words. For example, "embedded software engineer" keeps "Sr Embedded Engineer". Generic words such as engineer, senior or developer do not count (`TITLE_GENERIC_WORDS`). Dice and Monster use their own title search.

Duplicates. Each job gets a fingerprint built from its title, company, city and state. If the same job is already saved from another board, the new row is saved but flagged `is_duplicate` and pointed at the original through `canonical_job_id`. If the same board returns the same job again, the existing row is refreshed instead of a new row being added.

Data cleanup

Every board's data is cleaned into the same format before it is saved, so all jobs can be searched and filtered the same way.

Field	How it is cleaned
Location	Split into city, state and two-letter state code from text such as "San Francisco, CA", "New York, New York" or "Austin, TX 78701". ZIP codes and dots ("D.C.") are stripped. A location containing "remote" or "hybrid" sets those flags.
Salary	Numbers are read from text or numbers ("$85,000" → 85000); unreadable values are left empty. The pay period is standardized to yearly, hourly, daily, weekly or monthly.
Posting date	Invalid dates are left empty instead of being saved as bad values. Relative dates ("3 minutes ago") are converted to a real date.
Employment type	Standardized to lowercase forms such as full-time, contract and part-time; multiple types are kept, duplicates removed.
Company	Stored as posted, plus a normalized form used for duplicate detection and visa matching.

Some boards need specific handling:

Board	Handling
Monster	For a reposted job, the newer of its two date fields is used, so reposts are not treated as stale.
CareerBuilder	A "city" that is really a state name (multi-location jobs) is dropped. State names and codes both map to the code. US jobs with no country are accepted when the state is valid. Its many salary-unit spellings are standardized, and a hidden tracking image is removed from descriptions.
ZipRecruiter	Filler cards with no company are dropped. Clearly corrupted salaries above a per-period limit are removed. Foreign jobs (for example Toronto, ON) are dropped.

Board	Handling
JobRight	The board's salary, description and job-type fields come back empty, so salary and job type are taken from its requirements and benefits lists. No description is saved, because those lists mix in unrelated page text.
LinkedIn, Indeed, Glassdoor	One shared parser. Glassdoor returns at most about 28–30 jobs per search, so a per-board limit of 30 avoids a false shortfall.

Visa data loading (Component 1B)

Government files are downloaded by hand each quarter and placed in `data/`, because the DOL and USCIS download links change every release. `python manage.py visa_refresh` then builds one row per company in `company_visa_history`. It is run once a quarter after the new files are downloaded (by hand, Task Scheduler or cron); there is no scheduler inside the app. A missing file is skipped with a clear message and does not stop the others.

Source	Folder	What it fills	Key handling
DOL LCA disclosure (H-1B)	`data/lca`	H-1B filings count, median H-1B salary, last filing year	Only `VISA_CLASS = H-1B` rows count; E-3 and H-1B1 rows are excluded. Wages are converted to a yearly figure and unit typos are fixed before taking the median. The file is read row by row, so very large files load safely.
DOL PERM disclosure (green card)	`data/perm`	PERM filings count, last PERM year, green-card sponsor flag	Uses the post-June-2023 column `EMP_BUSINESS_NAME`, with the older `EMPLOYER_NAME` as a fallback

Source	Folder	What it fills	Key handling
USCIS H-1B Employer Data Hub	`data/uscis-hub`	H-1B petition approvals and denials	The only source for approvals and denials, which are real USCIS decisions. One employer's worksite rows are summed across all six petition types. Uses the .xlsx export.
E-Verify employer list (Open + Terminated)	`data/everify`	E-Verify enrolled: yes / no / unknown	All files in the folder are read together. Any Open account means enrolled; only Terminated accounts means not enrolled; not in the list means unknown, not "no". Account IDs and "and subsidiaries" wording are removed from names. Written in batches of 5,000.

One file is read per folder (the first by name), except E-Verify, where every file is read.

Replace, not add. DOL files are cumulative for the fiscal year, so each load replaces that source's numbers instead of adding to them. Loading the same file again never doubles the counts.

Company names. Every name is normalized before grouping: "Google LLC", "Google, Inc." and "GOOGLE" all become `google`. Only legal suffixes (Inc, LLC, Ltd, Corp ...) are removed; words such as Group, Software or Services are kept, because they often tell two companies apart. Each company therefore gets exactly one row, combined from all four sources.

Scoring. After all files load, every company is re-scored from its combined data:

Signal	Points
H-1B volume (the higher of LCA filings or USCIS decisions): 1+, 5+, 25+, 100+	10 / 20 / 30 / 40
USCIS approval rate	up to 25
Last filing this year or last year / within 3 years	15 / 8
Any PERM (green card) filing	10
E-Verify enrolled	10

The total is capped at 100 and graded A ⩾ 85, B ⩾ 70, C ⩾ 50, D ⩾ 30, E ⩾ 15, otherwise F.

After a visa refresh, `python manage.py run_match --rematch-all` re-matches every saved job so all jobs pick up the new grades.

Visa matching (Component 1C)

After every scrape, each job not yet matched is linked to one company in `company_visa_history`, and that company's visa data is copied onto the job. The rule throughout is to leave a job unmatched rather than guess, so the platform never shows a false "this company sponsors H-1B".

1. **Hand-checked aliases.** The `company_aliases` table maps a brand name seen in postings to the company's government filing name, for example "CGI Group" to "CGI Technologies and Solutions". This pass runs first so a verified mapping always wins.
2. **Exact name.** The job's normalized company name equals a company's normalized name. This covers most jobs.
3. **Fuzzy.** For names of 6 or more characters, the closest company with string similarity of at least 0.9 is accepted, but only if a second, independent letter-order check also scores at least 0.85.
4. **Prefix.** Names of at least 7 characters and 2 words are matched when one name starts with the other and exactly one company fits, for example "Pacific Life" and "Pacific Life Insurance". A name that fits two or more companies is left unmatched.
5. **Last resort.** The job is matched by exact name to a company that has an E-Verify record but no H-1B or PERM filings.

Passes 2–4 only match against companies with real sponsorship evidence (an LCA, USCIS or PERM filing). This stops a parent brand that appears only in E-Verify from claiming a job before its subsidiary, the actual H-1B filer, can be found.

What is copied onto the job: `visa_matched`, `h1b_sponsor`, `h1b_filings_recent` (the H-1B count if the company filed within the last 2 years, otherwise 0), `everify_enrolled`, `green_card_sponsor`, `visa_grade` and `visa_friendly_score`. These values are copied when a job is matched, so after a visa refresh `python manage.py run_match --rematch-all` re-matches every job.

Company aliases (added by hand)

Aliases are never created automatically. When a large company stays unmatched because its postings use a brand name ("Kforce Technology Staffing") and its filings use a legal name ("Kforce Inc"), someone checks the mapping (for example on myvisajobs.com) and adds it:

```
python manage.py add_alias "Kforce Technology Staffing" "Kforce" "same company"
python manage.py run_match
```

`add_alias` normalizes both names and checks that the filing name already exists in `company_visa_history`, so a typo fails instead of creating an alias that never matches. The alias is saved in the database straight away. To keep it for new installations as well, it is also added to the alias list at the end of `schema.sql`, which `setup_db` loads into every new database. That list currently holds 39 aliases, including the 4 added in October 2026 (Kforce, American Family Insurance, St. Jude Children's Research Hospital - ALSAC, UC Irvine). Some entries are "pins" (alias = target) that stop the fuzzy or prefix pass from matching an unrelated company.

Database tables and logging

The database is PostgreSQL with four tables, defined in `job_scraper/schema.sql`. Every SQL statement the app runs is kept in one file, `job_scraper/core/sql.py`, so a query change is made in one place.

Table	One row per	Written by	Main contents
`us_jobs`	Scraped job	Daily scrape, matcher, inactive check	Job details, parsed location and salary, source, posting date, duplicate flags, copied visa fields, active flag, first / last seen

Table	One row per	Written by	Main contents
`company_visa_history`	Company (normalized name)	Visa refresh	H-1B filings, approvals, denials, approval rate, median H-1B salary, last filing year, PERM filings, green-card flag, E-Verify, score, grade
`company_aliases`	Brand name → filing name	`add_alias` (by hand); 39 loaded by `setup_db`	Alias, target company and a note on how the mapping was confirmed
`scraping_log`	Board search, matcher run or visa refresh	Scraper, matcher, visa loader	Run type, board, role, start / end time, found / inserted / updated / duplicate counts, status, error, Apify run id, number of Apify runs, Apify cost

Indexes cover the filters a future search API will use: state, city, remote, salary, posting date, source, active, company, H-1B sponsor, visa score, plus full-text search on the job title.

Apify columns in `scraping_log`. Each board search can use several Apify runs (deeper pages and retries). Its row keeps:

Column	Holds
`apify_run_id`	The last Apify run of that search. Open it in the Apify Console to see the actor's log.
`apify_runs`	How many Apify runs the search used
`apify_cost_usd`	The total cost of those runs, in US dollars

Logging. Every board search for every role writes a `daily_scrape` row. Each matcher run writes a `match` row with the number of jobs matched, and each visa refresh writes a `visa_refresh` row marked success or failed. The console log also shows each Apify run's

input, run ID, status, row count, cost and Console link, and explains why a board returned nothing (blocked, no results, filtered or too old).

Data retention. The pipeline never deletes jobs or scraped data. Old jobs are only flagged inactive.

Django models. Read-only models are included for the project's own code and the future API: `UsJob`, `CompanyVisaHistory`, `CompanyAlias` and `ScrapingLog` (for example `UsJob.objects.filter(h1b_sponsor=True, state_code='NY')`). They are unmanaged, so Django never changes the tables; the tables come only from `schema.sql`.

Settings

All run settings live in `.env` (template: `job_scraper/env.example`), so limits, boards and actors can change without a code change. Values already set in the environment win over `.env`. The scrape settings can also be overridden for a single run on the command line.

Setting	Current value	Meaning
`SCRAPE_MAX_JOBS_PER_ROLE`	50	Jobs wanted per role
`SCRAPE_LIMIT_SCOPE`	per_board	`per_board` takes 50 from each board; `per_role` takes 50 in total across boards, in priority order
`SCRAPE_COUNT_MODE`	inserted	`inserted` counts only brand-new jobs; `all` also counts refreshed and duplicate jobs
`SCRAPE_BOARD_LIMITS`	(empty)	Per-board override, for example `glassdoor:30,dice:20`
`SCRAPE_SOURCES`	(empty = all boards)	Boards to run, in priority order; remove a board here to switch it off
`SCRAPE_FRESHNESS_DAYS`	1	Posting-age window (1 = last 24 hours)
`SCRAPE_KEEP_UNDATED`	true	Keep jobs with no posting date
`SCRAPE_INACTIVE_AFTER_DAYS`	45	Days unseen before a job is marked inactive

Setting	Current value	Meaning
`SCRAPE_MAX_FETCH_MULTIPLIER`	40	Cost ceiling: maximum raw rows per board per role, as a multiple of the limit
`SCRAPE_PARALLEL_BOARDS`	8	Board searches running at the same time (`per_board` scope; default 3 if not set)
`SCRAPE_SHARED_QUEUE`	true	`true` : all role × board searches share one queue. `false` : one role at a time, its boards in parallel.
`SCRAPE_RETRY_FAILED`	1	Retries for a blocked or broken board
`TITLE_MATCH_MODE`	distinctive	How strictly a job title must match the role (`distinctive` or `all`)
`TITLE_GENERIC_WORDS`	(empty = built-in list)	Words that do not count as distinctive, such as engineer, senior or developer
`MULTI_REQUIRE_TITLE` / `MULTI_SCAN_FACTOR`	true / 6	Title filter inside the multi-board actor, and how far ahead it looks (filtered rows are not billed)
`ZIPRECRUITER_EXACT_PHRASE` / `ZIPRECRUITER_US_ONLY`	true / true	ZipRecruiter phrase search and US-only filter
`MONSTER_LOCATION`	(empty = whole US)	Restrict Monster to a state or "City, ST"
`MUSE_LOCATIONS` / `MUSE_US_ONLY` / `MUSE_SCAN_PER_CITY`	3 cities / true / 20	The Muse search cities, US-only filter and look-ahead (only used if `muse` is in `SCRAPE_SOURCES`)
`APIFY_API_TOKEN`	(secret)	Apify account token
`APIFY_*_ACTOR`	one per board	Apify actor for each board (see Job scraping)
`APIFY_LOG_RUNS` / `APIFY_STREAM_LOGS`	true / false	Print each actor run's input, run ID, row count and cost / copy the actor's own log into ours

Setting	Current value	Meaning
`JOB_SCRAPER_DB_ALIAS`	default	Which `DATABASES` entry holds the tables (also accepted as a Django setting)
`JOB_SCRAPER_ENV_FILE`	(not set)	Load a different `.env` file, or set it empty to load none when the project manages environment variables itself
`DATA_DIR`	./data	Folder with the government files

`DATABASE_URL` and `DB_POOL_MAX` from the Node version are no longer used; the database connection comes from the Django project's `DATABASES` setting.

Commands

All commands run from the folder that holds `manage.py`, with the virtual environment active. They exit with code 0 when they finish and 1 on a bad setting or a failure.

Command	What it does	Node.js equivalent
`python manage.py setup_db`	Creates all tables and indexes, adds any new columns, and loads the 39 starting company aliases (safe to re-run)	`npm run setup-db`
`python manage.py daily_run`	Full daily run: scrape, match, mark inactive, summary	`npm run daily-run`
`python manage.py daily_run --check-config`	Shows settings and the cost estimate; scrapes nothing	`npm run daily-run ---check-config`
`python manage.py daily_run --each-board`	Small live test of every board	`npm run daily-run ---each-board`
`python manage.py peek_board --board dice --role "iot engineer" --n 20`	Shows the raw rows one board returns; saves nothing	`npm run peek-board -- ...`
`python manage.py visa_refresh`	Loads the government files in `data/` and re-scores every company (once a quarter)	`npm run visa-refresh`

Command	What it does	Node.js equivalent
`python manage.py run_match`	Re-runs the matcher on unmatched jobs; add `--rematch-all` to re-match every job	`npm run run-match`
`python manage.py add_alias "<brand>" "<filing name>" "<note>"`	Adds a verified company alias (the filing name must already exist)	`npm run add-alias -- ...`
`python manage.py verify_data`	Runs the read-only acceptance checks (counts, coverage, query speed)	`npm run verify-data`
`python manage.py test_board_parsers`	Offline checks of every board parser (no Apify, no database)	`npm run test-board-parsers`

`npm run build` has no Python equivalent; nothing needs compiling.

Command-line options for one run

These options change a single `daily_run` without editing `.env`. They go straight after the command (no `--` in front as with npm), for example `python manage.py daily_run --file roles-data.txt --max 5`. The run prints where each setting came from: command line, `.env` or default.

Option	Overrides	Example use
`--file <path>`	Roles file (default `roles.txt`)	Switch job families: `--file roles-java.txt`; a JSON array or `{ "roles": [...] }` also works
`--max <N>`	`SCRAPE_MAX_JOBS_PER_ROLE`	Cheap test run: `--max 5`
`--days <N>`	`SCRAPE_FRESHNESS_DAYS`	Widen the window to 2 days
`--scope per_board|per_role`	`SCRAPE_LIMIT_SCOPE`	Take 50 per role in total instead of 50 per board
`--count inserted|all`	`SCRAPE_COUNT_MODE`	Count refreshed and duplicate jobs too
`--sources a,b`	`SCRAPE_SOURCES`	Run only some boards: `--sources linkedin,dice`

Option	Overrides	Example use
`--board-limits b:N,...`	`SCRAPE_BOARD_LIMITS`	`--board-limits glassdoor:30,dice:20`
`--no-match`	—	Skip the visa matcher for this run
`--no-inactive`	—	Skip marking stale jobs inactive
`--check-config`	—	Show settings and cost estimate only; never scrapes, even with `--each-board`
`--each-board`	—	Test every board with 3 jobs (or `--max`)
`--help`	—	Print usage

Scheduling

`daily_run` is started once a day by an outside scheduler; there is no scheduler inside the app. `visa_refresh` is run once a quarter after new government files are downloaded, followed by `run_match --rematch-all`.

- **Windows (Task Scheduler):** a task that runs `C:\path\to\venv\Scripts\python.exe manage.py daily_run`, with "Start in" set to the project folder.
- **Linux (cron):**

```
15 6 * * *  cd /path/to/project && /path/to/venv/bin/python manage.py daily_run >> logs/daily-run.log 2>&1
```

Code layout

Everything lives in one Django app folder, `job_scraper/`, inside the project. The code keeps short comments only where a rule is not obvious from the code itself (for example why DOL loads replace instead of add, or why JobRight runs without a proxy); `README.md` in the same folder covers setup and commands.

Path	Contents	Node.js original
`management/commands/`	The `manage.py` commands (`daily_run`, `visa_refresh`, `run_match`, `add_alias`, `peek_board`, `verify_data`, `setup_db`, `test_board_parsers`)	`scripts/*.ts`
`config/`	Board list and scrape settings, with validation	`src/config/`
`core/`	Database access, all SQL (`sql.py`), logging, roles file, JavaScript-compatible helpers, settings access	`src/db/`, `src/roles/`
`scraper/`	Apify client, one parser per board, the scrape loop (`job_scraper.py`)	`src/scraper/`
`visa/`	Government file loaders (LCA, PERM, USCIS Hub, E-Verify) and scoring	`src/visa/`
`matcher/`	Company name normalizing, string similarity and the visa matcher	`src/matcher/`
`jobs/`	Daily role scrape, quarterly visa refresh, mark inactive	`src/jobs/`
`schema.sql`	Table definitions and the starting aliases	`schema.sql`
`models.py`, `routers.py`, `migrations/`, `apps.py`	Django integration (read-only models, optional database router, table setup)	(new)
`env.example`, `requirements.txt`, `README.md`	Settings template, Python packages, setup and command guide	`.env.example`, `package.json`, `README.md`

`core/jscompat.py` keeps results identical to the original TypeScript code where Python and JavaScript differ (rounding of halves, reading dates and numbers, regular expressions).

Calling it from Python (for the API phase)

The commands are thin wrappers, so an API or a background task can call the same functions:

```
from job_scraper.config.scrape_config import load_scrape_config,
ScrapeConfigOverrides
from job_scraper.core.roles import distinct_roles
from job_scraper.jobs.role_scrape import run_role_scrape
from job_scraper.jobs.mark_inactive import mark_stale_jobs_inactive

cfg = load_scrape_config(ScrapeConfigOverrides(max='20',
sources='linkedin,dice'))
roles = distinct_roles(['data analyst', 'iot engineer']).roles
summary = run_role_scrape(roles, cfg)        # scrape + visa matcher
mark_stale_jobs_inactive(cfg.inactiveAfterDays)
```

A run takes minutes to hours, so an API should start it in the background and return straight away. `core.log.set_streams(stdout=..., stderr=...)` sends a run's log somewhere else, for example a file per run.

Cost

Apify bills every row an actor returns, not only the new jobs we keep. A daily run's cost is therefore set by the rows returned per board, and the code limits that in four ways:

- Each board is asked for only the jobs still needed.
- On LinkedIn, Indeed, Glassdoor and the remote boards, the title filter runs inside the actor, so off-title rows are never delivered or billed.
- ZipRecruiter is searched as an exact phrase, because a loose search returned mostly unrelated jobs.
- A hard ceiling caps raw rows per board per role at `SCRAPE_MAX_FETCH_MULTIPLIER` × the board's limit.

Before spending anything, `python manage.py daily_run --check-config` prints the run size and the worst-case billed rows.

Board	Actor pricing
CareerBuilder	About $2 per 1,000 rows (descriptions included)
ZipRecruiter	About $0.99 per 1,000 rows, as tested

Board	Actor pricing
LinkedIn, Indeed, Glassdoor, remote boards	Pay per result on the multi-board actor; filtered rows are not billed
Dice, Monster, JobRight	Pay per result; the current rate is on each actor's Apify page

Worked example (current settings). `roles.txt` has 8 roles (the data analyst family). `SCRAPE_SOURCES` is empty, so all 15 boards run: the 8 required boards plus 7 optional remote boards.

Measure	Calculation	Rows
Jobs asked for per run	8 roles × 15 boards × 50	6,000
Worst-case billed rows (ceiling)	6,000 × 40 fetch multiplier	240,000
Same run, required 8 boards only	8 roles × 8 boards × 50	3,200 asked; 128,000 ceiling

Real runs bill far fewer rows than the ceiling, because a board that runs out of fresh 24-hour jobs stops early, and only rows actually returned are billed.

Fetch multiplier. It is set to 40 in `.env`, so a board in `inserted` mode can page through up to 2,000 rows to find 50 new jobs. CareerBuilder asks only for its 50-row limit in a single call. ZipRecruiter also runs as a single call, but that call is sized to the whole fetch budget (50 × 40 = 2,000 rows).

Memory. Each running actor uses some of the Apify account's memory, which limits how many searches can run at once (`SCRAPE_PARALLEL_BOARDS`). If runs fail with a memory-limit error, lower that number.

How cost is tracked

Every Apify run's cost is read back from Apify when the run finishes:

- The cost is Apify's own figure for the run (`usageTotalUsd`), or the actor's charges for the run (price per result or per event × how many) when those are higher, because Apify adds the actor's charges to its figure a little after the run ends.
- The console log prints each run's cost next to its row count, for example `SUCCEEDED, 30 row(s), $0.1350`.
- Each board search stores its run count and total cost in `scraping_log` (`apify_runs`, `apify_cost_usd`).

- The end-of-run status table has a `cost $` column, the per-board totals show each board's cost, and the last line gives the whole run: `Apify cost for this run: $X (N actor runs)`.

Changes from the handoff document (v1.2)

The build follows the handoff's three components and table design. These are the places where it differs, each made for a tested reason.

Area	Handoff v1.2	Built	Why
Platform	Node.js service	Django app inside the client's existing Django project, run with `python manage.py`	The client runs Django and does not want a separate server for the scraper
Search terms	Fixed list of 20 tech queries in code	Roles file (`roles.txt` / `.json`), duplicates removed	The client's role list changes; currently the data analyst family
Volume	500 jobs per query, 14-day window	50 new jobs per role per board, 24-hour window, all set in `.env`	Client requirement for fresh daily jobs; controls Apify cost
Boards	LinkedIn, Indeed, Dice, Glassdoor, ZipRecruiter, Google Jobs	LinkedIn, Indeed, Glassdoor, Dice, Monster, CareerBuilder, ZipRecruiter, JobRight + optional remote boards	Client's required list; a separate parser per board
Actors	Default actor IDs in code	Every actor only in `.env` ; missing actor = clear skip message	Swapping an actor is a config change
Title quality	Not covered	Title must contain the role's distinctive words	Loose board searches returned many unrelated jobs, and every row is billed

Area	Handoff v1.2	Built	Why
Parallel scraping	Not covered	Up to `SCRAPE_PARALLEL_BOARDS` searches at once, optionally in one shared queue across roles	Shorter daily runs
Cost tracking	Not covered	Apify run id, run count and cost per search in `scraping_log`; run total in the summary	Cost can be checked per board and per run
Daily scheduling	node-cron inside the app	`python manage.py daily_run` started by an outside scheduler (Task Scheduler / cron); the quarterly visa refresh is a command too	Easier to monitor and re-run
Stale jobs	Optional	Inactive after 45 days unseen; rows kept	Client rule: never delete scraped data
H-1B approvals	Same loader as LCA filings	Only from the USCIS Data Hub	LCA "Certified" is a wage certification, not a visa approval
H-1B salary	Average	Median, after fixing wage-unit typos	Typos produced averages above $1M
Visa loads	Counts added on each load	Each source's numbers replaced	DOL files are cumulative; adding doubled the counts
E-Verify	Yes / no	Yes / no / unknown	Large companies are often enrolled under a parent name
Recent filings	`h1b_filings_2024` + `h1b_filings_2025`	Full count if last filed within 2 years	The year columns were never filled; they have been removed

Area	Handoff v1.2	Built	Why
Matching	Exact match, optional fuzzy	Alias → exact → fuzzy → prefix → last-resort exact	Higher coverage while still refusing weak guesses
SQL	Inline in each file	All in `core/sql.py`	One place to change queries